ASSIGNMENT 17.5

2303A51417
Cherepally Akhila
Batch 20

Task 1 –Loan Approval Dataset Cleaning
PROMPT:
#Write Python code using pandas and scikit-learn to preprocess a loan approval dataset loaded from a CSV file named "loanapproval.csv". The preprocessing should include handling missing values in the columns loan_amount and credit_score by filling them with appropriate statistical values such as the mean or median. Encode the categorical column loan_status into numerical form (Approved = 1, Rejected = 0). Identify and scale all numerical features using StandardScaler for better model performance. Finally, split the dataset into training and testing sets using train_test_split with an appropriate test size. Ensure the code is clean, well-structured, and prints sample outputs after each preprocessing step for verification.
#you onli create a sampledataset for the demonstration purpose, you can use the full dataset in your actual implementation.

CODE:

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Sample dataset
data = {
    'loan_amount': [10000, 15000, None, 20000, 25000],
    'credit_score': [700, 750, 720, None, 680],
    'loan_status': ['Approved', 'Rejected', 'Approved', 'Rejected', 'Approved']
}

df = pd.DataFrame(data)

print("📊 Original Data:")
print(df)

# Handle missing values
df['loan_amount'] = df['loan_amount'].fillna(df['loan_amount'].mean())
df['credit_score'] = df['credit_score'].fillna(df['credit_score'].median())

print("\n✅ After handling missing values:")
print(df)

# Encode loan_status
df['loan_status'] = df['loan_status'].map({'Approved': 1, 'Rejected': 0})

print("\n🔢 After encoding loan_status:")
print(df)
```

```python
# Separate features and target
X = df[['loan_amount', 'credit_score']]
y = df['loan_status']

# Scale numerical features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("\n🏷️ Scaled Features:")
print(X_scaled)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

print("\n📦 Training Data:")
print(X_train)
print(y_train)

print("\n🧪 Testing Data:")
print(X_test)
print(y_test)
```

OUTPUT:

```
📊 Original Data:
   loan_amount  credit_score loan_status
0      10000.0         700.0    Approved
1      15000.0         750.0    Rejected
2          NaN         720.0    Approved
3      20000.0           NaN    Rejected
4      25000.0         680.0    Approved

✅ After handling missing values:
   loan_amount  credit_score loan_status
0      10000.0         700.0    Approved
1      15000.0         750.0    Rejected
2      17500.0         720.0    Approved
3      20000.0         710.0    Rejected
4      25000.0         680.0    Approved
```

```
🔢 After encoding loan_status:
   loan_amount  credit_score  loan_status
0      10000.0         700.0            1
1      15000.0         750.0            0
2      17500.0         720.0            1
3      20000.0         710.0            0
4      25000.0         680.0            1

📏 Scaled Features:
[[-1.5        -0.51832106]
 [-0.5         1.64135001]
 [ 0.          0.34554737]
 [ 0.5        -0.08638684]
 [ 1.5        -1.38218948]]
```

```
📦 Training Data:
[[ 1.5        -1.38218948]
 [ 0.          0.34554737]
 [-1.5        -0.51832106]
 [ 0.5        -0.08638684]]
4    1
2    1
0    1
3    0
Name: loan_status, dtype: int64

🧪 Testing Data:
[[-0.5         1.64135001]]
1    0
Name: loan_status, dtype: int64
PS C:\Users\akhil\OneDrive\Desktop\Ai asst Coding>
```

Task 2 – Social Media Profiles

PROMPT:
Write Python code using pandas and scikit-learn to preprocess a social media engagement dataset. For demonstration purposes, first create a sample dataset that includes columns such as user_id, username, age, gender, platform, posts_count, likes, and engagement_level. Perform preprocessing steps including removing duplicate user profiles based on user_id, handling missing values in numerical and categorical columns using appropriate techniques (mean/median for numerical and mode for categorical), and encoding categorical columns like gender, platform, and engagement_level into numerical form. Ensure the code is clean, well-structured, and prints the dataset before and after each preprocessing step for clear verification.

CODE:

```
import pandas as pd
# Sample dataset
data = {
    'user_id': [1, 2, 3, 4, 5, 1],  # Duplicate user_id
    'username': ['user1', 'user2', 'user3', 'user4', 'user5', 'user1'],
    'age': [25, 30, None, 22, 28, 25],
    'gender': ['Male', 'Female', 'Female', None, 'Male', 'Male'],
    'platform': ['Instagram', 'Facebook', 'Twitter', 'Instagram', None, 'Instagram'],
    'posts_count': [100, 150, 200, None, 250, 100],
    'likes': [1000, 1500, None, 2000,   2500, 1000],
    'engagement_level': ['High', 'Medium', 'Low', 'Medium', None   ,
                            'High']
}
df = pd.DataFrame(data)
print("📊 Original Data:")
print(df)
# Remove duplicate user profiles based on user_id
df = df.drop_duplicates(subset='user_id')
print("\n✅ After removing duplicates:")
print(df)
# Handle missing values in numerical columns
df['age'] = df['age'].fillna(df['age'].mean())
df['posts_count'] = df['posts_count'].fillna(df['posts_count'].mean())
df['likes'] = df['likes'].fillna(df['likes'].mean())
print("\n✅ After handling missing values in numerical columns:")
print(df)
```

OUTPUT:

```
📊 Original Data:
   user_id username   age  gender   platform  posts_count   likes engagement_level
0        1    user1  25.0    Male  Instagram        100.0  1000.0             High
1        2    user2  30.0  Female   Facebook        150.0  1500.0           Medium
2        3    user3   NaN  Female    Twitter        200.0     NaN              Low
3        4    user4  22.0     NaN  Instagram          NaN  2000.0           Medium
4        5    user5  28.0    Male        NaN        250.0  2500.0              NaN
5        1    user1  25.0    Male  Instagram        100.0  1000.0             High

✅ After removing duplicates:
   user_id username   age  gender   platform  posts_count   likes engagement_level
0        1    user1  25.0    Male  Instagram        100.0  1000.0             High
1        2    user2  30.0  Female   Facebook        150.0  1500.0           Medium
2        3    user3   NaN  Female    Twitter        200.0     NaN              Low
3        4    user4  22.0     NaN  Instagram          NaN  2000.0           Medium
4        5    user5  28.0    Male        NaN        250.0  2500.0              NaN
```

```
4        5    user5  28.0    Male        NaN        250.0  2500.0              NaN

✅ After handling missing values in numerical columns:
   user_id username    age  gender   platform  posts_count   likes engagement_level
0        1    user1  25.00    Male  Instagram        100.0  1000.0             High
1        2    user2  30.00  Female   Facebook        150.0  1500.0           Medium
2        3    user3  26.25  Female    Twitter        200.0  1750.0              Low
3        4    user4  22.00     NaN  Instagram        175.0  2000.0           Medium
4        5    user5  28.00    Male        NaN        250.0  2500.0              NaN
PS C:\Users\akhil\OneDrive\Desktop\Ai asst Coding>
```

Task 3 – Weather Dataset Feature Engineering

PROMPT:
Write Python code using pandas and scikit-learn to preprocess a weather dataset for forecasting purposes. For demonstration, first create a sample dataset containing columns such as date, temperature, humidity, and rainfall. Convert the date column into datetime format for proper time-based analysis. Handle missing values in temperature and humidity using appropriate techniques like mean or median imputation. Perform feature engineering by creating new columns such as season (based on month) and week_number extracted from the date. Normalize the rainfall values using a suitable scaling technique such as MinMaxScaler or StandardScaler. Ensure the code is clean, well-structured, and prints the dataset before and after each preprocessing step for verification.

CODE:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
# Sample dataset
data = {
    'date': ['2024-01-01', '2024-02-01', '2024-03-01', '2024-04-01', '2024-05-01'],
    'temperature': [30, 28, None, 25, 22],
    'humidity': [80, None, 75, 70, 65],
    'rainfall': [5, 10, 15, None, 20]
}
df = pd.DataFrame(data)
print("📊 Original Data:")
print(df)
# Convert date column to datetime format
df['date'] = pd.to_datetime(df['date'])
print("\n✅ After converting date to datetime format:")
print(df)
# Handle missing values in temperature and humidity
df['temperature'] = df['temperature'].fillna(df['temperature'].mean())
df['humidity'] = df['humidity'].fillna(df['humidity'].mean())
print("\n✅ After handling missing values in temperature and humidity:")
print(df)
# Feature engineering: Create season and week_number columns

df['season'] = df['date'].dt.month % 12 // 3 + 1  # 1: Winter, 2: Spring, 3: Summer, 4:
Fall
df['week_number'] = df['date'].dt.isocalendar().week
```

```
print("\n✅ After feature engineering:")
print(df)
# Normalize rainfall values using MinMaxScaler
scaler = MinMaxScaler()
df['rainfall_normalized'] = scaler.fit_transform(df[['rainfall']])
print("\n✅ After normalizing rainfall values:")
print(df)
```

OUTPUT:

```
📊 Original Data:
         date  temperature  humidity  rainfall
0  2024-01-01         30.0      80.0       5.0
1  2024-02-01         28.0       NaN      10.0
2  2024-03-01          NaN      75.0      15.0
3  2024-04-01         25.0      70.0       NaN
4  2024-05-01         22.0      65.0      20.0

✅ After converting date to datetime format:
        date  temperature  humidity  rainfall
0 2024-01-01         30.0      80.0       5.0
1 2024-02-01         28.0       NaN      10.0
2 2024-03-01          NaN      75.0      15.0
3 2024-04-01         25.0      70.0       NaN
4 2024-05-01         22.0      65.0      20.0
```

```
✅ After handling missing values in temperature and humidity:
        date  temperature  humidity  rainfall
0 2024-01-01        30.00      80.0       5.0
1 2024-02-01        28.00      72.5      10.0
2 2024-03-01        26.25      75.0      15.0
3 2024-04-01        25.00      70.0       NaN
4 2024-05-01        22.00      65.0      20.0

✅ After feature engineering:
        date  temperature  humidity  rainfall  season  week_number
0 2024-01-01        30.00      80.0       5.0       1            1
1 2024-02-01        28.00      72.5      10.0       1            5
2 2024-03-01        26.25      75.0      15.0       2            9
3 2024-04-01        25.00      70.0       NaN       2           14
4 2024-05-01        22.00      65.0      20.0       2           18
```

```
✅ After normalizing rainfall values:
        date  temperature  humidity  rainfall  season  week_number  rainfall_normalized
0 2024-01-01        30.00      80.0       5.0       1            1             0.000000
1 2024-02-01        28.00      72.5      10.0       1            5             0.333333
2 2024-03-01        26.25      75.0      15.0       2            9             0.666667
3 2024-04-01        25.00      70.0       NaN       2           14                  NaN
4 2024-05-01        22.00      65.0      20.0       2           18             1.000000
```

Task 4 – Movie Ratings Dataset Preparation

PROMPT:

Write Python code using pandas and scikit-learn to clean and preprocess a movie ratings dataset. For demonstration purposes, first create a sample dataset that includes columns such as movie_id, title, genre, language, and rating. Perform preprocessing steps including handling missing values in rating and genre using appropriate techniques (such as mean for rating and mode for genre), encoding categorical variables like genre and language into numerical form using suitable encoding methods, and removing duplicate movie entries based on movie_id. Finally, scale the rating values between 0 and 1 using a normalization technique such as MinMaxScaler. Ensure the code is clean, well-structured, and prints the dataset before and after each preprocessing step for clear verification.

CODE:

```
import pandas as pd

from sklearn.preprocessing import MinMaxScaler
# Sample dataset
data = {
    'movie_id': [1, 2, 3, 4, 5, 1],  # Duplicate movie_id
    'title': ['Jersey', 'Eega', 'Pokiri', 'ok jaanu', 'Dhurandhar', 'HI nanna'],
    'genre': ['Family', 'Drama', None, 'Rom-Com', 'Action', 'Family'],
    'language': ['Telugu', 'Telugu', 'Telugu', None, 'Telugu', 'Telugu'],
    'rating': [4.5, None, 3.0, 4.0, 5.0, 4.5]
}
df = pd.DataFrame(data)
print("📊 Original Data:")
print(df)
# Remove duplicate movie entries based on movie_id
df = df.drop_duplicates(subset='movie_id')
print("\n✅ After removing duplicates:")
print(df)
# Handle missing values in rating and genre
df['rating'] = df['rating'].fillna(df['rating'].mean())
df['genre'] = df['genre'].fillna(df['genre'].mode()[0])
print("\n✅ After handling missing values in rating and genre:")
print(df)

# Encode categorical variables genre and language
df['genre_encoded'] = df['genre'].astype('category').cat.codes
df['language_encoded'] = df['language'].astype('category').cat.codes
print("\n✅ After encoding genre and language:")
print(df)
# Scale rating values between 0 and 1 using MinMaxScaler
scaler = MinMaxScaler()
df['rating_normalized'] = scaler.fit_transform(df[['rating']])
print("\n✅ After normalizing rating values:")
print(df)
```

OUTPUT:

```
📊 Original Data:
   movie_id       title    genre language  rating
0         1      Jersey   Family   Telugu     4.5
1         2        Eega    Drama   Telugu     NaN
2         3      Pokiri      NaN   Telugu     3.0
3         4    ok jaanu  Rom-Com      NaN     4.0
4         5  Dhurandhar   Action   Telugu     5.0
5         1    HI nanna   Family   Telugu     4.5

✅ After removing duplicates:
   movie_id       title    genre language  rating
0         1      Jersey   Family   Telugu     4.5
1         2        Eega    Drama   Telugu     NaN
2         3      Pokiri      NaN   Telugu     3.0
3         4    ok jaanu  Rom-Com      NaN     4.0
4         5  Dhurandhar   Action   Telugu     5.0
```

```
✅ After handling missing values in rating and genre:
   movie_id       title    genre language  rating
0         1      Jersey   Family   Telugu   4.500
1         2        Eega    Drama   Telugu   4.125
2         3      Pokiri   Action   Telugu   3.000
3         4    ok jaanu  Rom-Com      NaN   4.000
4         5  Dhurandhar   Action   Telugu   5.000

✅ After encoding genre and language:
   movie_id       title    genre language  rating  genre_encoded  language_encoded
0         1      Jersey   Family   Telugu   4.500              2                 0
1         2        Eega    Drama   Telugu   4.125              1                 0
2         3      Pokiri   Action   Telugu   3.000              0                 0
3         4    ok jaanu  Rom-Com      NaN   4.000              3                -1
4         5  Dhurandhar   Action   Telugu   5.000              0                 0
```

```
✅ After normalizing rating values:
   movie_id       title    genre language  rating  genre_encoded  language_encoded  rating_normalized
0         1      Jersey   Family   Telugu   4.500              2                 0             0.7500
1         2        Eega    Drama   Telugu   4.125              1                 0             0.5625
2         3      Pokiri   Action   Telugu   3.000              0                 0             0.0000
3         4    ok jaanu  Rom-Com      NaN   4.000              3                -1             0.5000
4         5  Dhurandhar   Action   Telugu   5.000              0                 0             1.0000
```